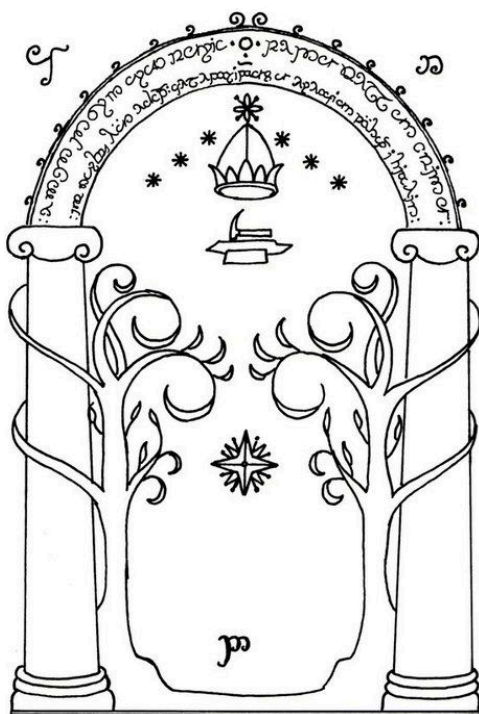




Proyecto Especial

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

6 de agosto de 2026

1. Contexto	2
2. Lineamientos	2
2.1. Equipo	2
2.2. Fuente	2
2.3. Comunicación	3
2.4. Entregables	3
2.5. Recursantes	3
3. Stage I: Diseño	4
3.1. Definición	4
3.2. FAQ	4
4. Stage II: Frontend	5
4.1. Definición	5
4.2. FAQ	6
5. Stage III: Backend	6
5.1. Definición	6
5.2. FAQ	7

1. Contexto

El conglomerado ELVEN DOOR desea incrementar la escalabilidad de sus desarrollos significativamente, pero debido a la inmensa cantidad de equipos de ingeniería y a la multiplicidad de tecnologías utilizadas, debe hallar una solución ubicua.

El Arquitecto determinó que la solución al problema consiste en implementar un DSL (*Domain Specific Language*), junto con su compilador, con lo cual, todos los equipos desarrollarán cada aplicación sobre dicho lenguaje, permitiendo que la innovación se produzca en simultáneo sobre todo el ecosistema del conglomerado cada vez que se introduzca un cambio sobre el compilador y el DSL asociado, aumentando la DX (*Developer Experience*).

Como efecto secundario, el equipo que desarrolle la solución, podrá integrar mejoras de rendimiento y seguridad automáticamente, al desplegar una nueva versión del compilador.

2. Lineamientos

2.1. Equipo

Para desarrollar dicho DSL, el Arquitecto solicita que se conforme un equipo de 1 (uno) a 4 (cuatro) integrantes, máximo:

1	2	3	4
HARDCORE	BRO-FORCE	DELTA-FORCE	RAID TEAM

2.2. Fuente

La base de código deberá ser versionada en Azure Repos, BitBucket, GitHub o GitLab. Deberá utilizar como base el proyecto [Flex-Bison-Compiler](#) (branch [development](#)), y se deberá desarrollar en lenguaje C haciendo uso de las herramientas Flex (analizador léxico), y Bison (analizador sintáctico).

En caso de que el equipo de desarrollo desee utilizar un repositorio privado, deberá solicitar al QRF (ver **Sección § 2.3: Comunicación**), la lista de nombres de usuario a los que deberá otorgarle permisos de solo-lectura.

Al clonar el repositorio por primera vez, deberán modificar su licencia en el archivo **LICENSE.md**, y ajustar las medallas (*badges*), disponibles en las primeras líneas de **README.md**, de modo que apunten al repositorio de desarrollo correcto y no al original.

2.3. Comunicación

En todo momento, las consultas, entregas y amenazas deberán ser dirigidas al QRF (Quick Response Force), vía correo electrónico con copia a todas las direcciones:

- (I). mdiaznoia@itba.edu.ar (Mariana Lucía Díaz Noia)
- (II). johuerta@itba.edu.ar (Joaquín José Huerta)
- (III). slifischtz@itba.edu.ar (Santiago Lifischtz)
- (IV). lbagnascomuguillo@itba.edu.ar (Lautaro Bagnasco Muguillo)
- (V). mgolmar@itba.edu.ar (Agustín Golmar)

2.4. Entregables

El proceso de desarrollo se extiende durante 3 meses según se indica en el documento disponible en  (2026-07-21) Cronograma, y se reparte en 3 entregas:

- (Stage I). **Diseño:** Solo involucra la confección de un documento inicial en formato PDF con requerimientos, propuesta, ejemplos y casos de prueba.
- (Stage II). **Frontend:** La construcción de un analizador léxico y sintáctico mediante Flex y Bison para el DSL propuesto, y un conjunto de programas de prueba. Dado un programa en la entrada, la aplicación debe lograr la construcción de su AST (*Árbol de Sintaxis Abstracta*).
- (Stage III). **Backend:** Un compilador funcional para el DSL que pasa correctamente todos los programas de prueba, y un documento PDF adicional con las especificaciones del desarrollo completo a lo largo de todo el proyecto.

Todas las entregas se deben versionar por completo en el repositorio donde se desarrolló el proyecto sobre la rama **development**, y se deben anunciar antes de que finalice el último día de cada semana de entrega por correo al QRF. El anuncio debe indicar:

- (I). Lista de integrantes, nombre o número de equipo¹.
- (II). *Commit hash* completo (40 caracteres), de la entrega sobre la rama **development**.
- (III). Link al repositorio, en caso de que se haya modificado desde la entrega anterior.

Cualquier detalle adicional, se deberá indicar en el README principal del repositorio, y no en el correo electrónico con el que se realiza la entrega (i.e., el repositorio debe ser **autocontenido**).

2.5. Recursantes

Si el Proyecto Especial fue realizado y aprobado en un período anterior, el alumno podrá optar por las siguientes opciones:

¹ El número de equipo es otorgado por el QRF luego de la primera entrega.

- (I). Mantener la misma nota final del período anterior.
- (II). Aumentar la nota anterior reparando todos los errores reportados por la cátedra.
- (III). Rehacer el mismo trabajo nuevamente.
- (IV). Rehacer el Proyecto Especial completo, individualmente, sobre otro DSL y su compilador.
- (V). Rehacer el Proyecto Especial como si fuera la primera vez que lo hace, en grupo.

Para cualquiera de las opciones seleccionadas (salvo en la Opción V), la fecha límite de entrega es única y se corresponde con la misma del Stage III. El recurrente debe seleccionar una de las opciones y reportarla lo antes posible al QRF.

3. Stage I: Diseño

3.1. Definición

El equipo deberá concebir y describir un dominio a elección, para el cual se construirá un lenguaje con el cual manipular y simular su comportamiento. El dominio seleccionado puede ser nuevo, o puede estar relacionado a lenguajes implementados previamente como aquellos disponibles en el documento [\(2026-09-22\) Proyectos Anteriores](#).

Para el dominio seleccionado, deberá definir las abstracciones necesarias y su representación (i.e., su sintaxis), dentro de un lenguaje formal basado, pero no limitado, en los conceptos definidos en [\(2026-03-12\) Modelo Computacional](#).

Finalmente, el equipo deberá completar una copia de la especificación de ejemplo [\(2026-09-18\) Especificación](#), indicando:

- (I). Integrantes del equipo.
- (II). Repositorio dónde será versionada la solución y su documentación.
- (III). Descripción del dominio seleccionado.
- (IV). Construcciones, prestaciones y funcionalidades del lenguaje propuesto a implementar.
- (V). Casos de prueba; mínimo 10 (diez) casos de **aceptación** y 5 (cinco) de **rechazo**.
- (VI). Al menos 2 (dos) ejemplos de sintaxis con una breve descripción.

La misma se debe versionar en el repositorio y en formato PDF, dentro de la carpeta **doc** en la raíz del proyecto. Luego, se deberá anunciar por correo electrónico tal cual lo indica la **Sección § 2.4: Entregables**.

3.2. FAQ

¿La sintaxis propuesta debe ser final o es modificable en subsiguientes entregas?

Es modificable, pero se debe tener en cuenta que el Stage II implica la construcción del AST para la misma, y si luego de la primera entrega la sintaxis se aleja significativamente de su forma final, entonces se perderá tiempo valioso para completar la siguiente etapa (*frontend*).

¿Qué tan complejo debe ser un caso o programa de prueba?

Debe ser un *Unit Test*, es decir, debe ser lo más compacto y específico posible para el caso de uso o *feature* del lenguaje que se esté probando. Si una línea es suficiente, entonces deberá ser de ese tamaño. Esto no quita que se puedan agregar casos/programas adicionales más complejos, pero los tests de unidad son obligatorios.

¿Qué sucede si la entrega no se realiza antes de la fecha estipulada?

La nota del trabajo comienza a decaer exponencialmente como un relámpago.

¿Por qué se solicita el *commit hash*?

Porque de esa manera el equipo puede seguir desarrollando libremente mientras el QRF evalúa la solución entregada, y el proceso no se detiene a la espera del *feedback*. Si luego de entregar, el equipo ya conoce aspectos que debe corregir para la siguiente entrega, no es necesario que espere el aval o la indicación del QRF; de esta manera el desarrollo es realmente continuo.

¿Qué sucede si la sintaxis o las prestaciones del lenguaje no se logran completar a tiempo?

Si bien lo ideal es que el proyecto mantenga un nivel de complejidad aceptable (que no sea trivial, pero tampoco imposible), a veces es difícil estimar la misma, sobre todo frente a una tecnología desconocida, y sobre un período extenso de tiempo (i.e., un cuatrimestre). Dicho esto, es preferible que sobren *features* y no que falten. Es preferible que el Stage II (*frontend*), posea la sintaxis deseada o ideal, pero que luego el Stage III (*backend*), solo implemente la parte fundamental.

4. Stage II: Frontend

4.1. Definición

Luego de presentar la especificación del lenguaje propuesto, se procederá a completar el diseño final de su sintaxis y a construir la solución. Para ello, se debe comenzar por establecer la definición de una gramática $G = \langle \Sigma, N, \Pi, S \rangle$, donde Σ es el alfabeto, N es el conjunto de símbolos no-terminales, Π es el conjunto de producciones libres de contexto (tipo 2, según la Jerarquía de Chomsky), y S es el símbolo no-terminal inicial de la gramática.

La solución a entregar debe:

- (I). Transformar un programa de entrada en el lenguaje desarrollado en un *stream* de *tokens* según se describe en el documento [\[2025-09-03, v1.0.0\] Análisis Léxico](#).
- (II). Transformar el *stream* de *tokens* en un Árbol de Sintaxis Abstracta (AST), según lo expuesto en el documento [\[2024-05-08, v0.1.0\] Análisis Sintáctico](#).
- (III). Permitir la correcta ejecución del *script* de *testing* disponible en el repositorio de base, aceptando o rechazando cada uno de los casos de prueba según corresponda.

Nótese que el analizador léxico será el que defina el alfabeto Σ , mientras que el analizador sintáctico es quien define N , Π y S , es decir, ambas fases conforman la gramática G que genera el lenguaje a implementar.

4.2. FAQ

¿Se llama *frontend* porque tiene UI?

No, para nada. Se llama *frontend* porque representa “el frente” del compilador, y se compone del analizador léxico y sintáctico. Todo lo demás, se denomina *backend*. El entregable produce un ejecutable que puede utilizarse por consola como cualquier *script* o comando; no necesita de ninguna interfaz gráfica.

Si todavía no hay *backend*, ¿cómo acepto o rechazo los casos de prueba?

Debido a que en esta segunda entrega, la fase de análisis semántico se encuentra incompleta, es probable que no sea posible aceptar o rechazar alguno de los casos de prueba propuestos, pero esto es completamente normal. Por ejemplo, un caso de prueba de asignación de variables con tipos incompatibles debería ser rechazado, pero como todavía no se implementa un mecanismo de *type-checking*, se deberá aceptar momentáneamente (*falso positivo*), salvo que la misma gramática provea construcciones con tipos bien formados naturalmente.

¿Sobre qué plataforma se realizará la construcción y prueba de la solución?

Es irrelevante ya que el proyecto base se desarrollará sobre un clúster de Docker Compose, dentro de una imagen de Ubuntu. Esto garantiza que todos los equipos desarrollen su solución en el mismo sistema operativo, pero sobre el entorno que cada uno posea; solo se requiere la dependencia Docker. En cualquier caso, se recomienda probar la solución y su construcción en uno o varios entornos antes de cada entrega, para garantizar la reproducibilidad de su instalación y ejecución².

5. Stage III: Backend

5.1. Definición

La etapa final del proyecto involucra completar el *frontend* del compilador junto con las siguientes fases:

- (I). Una fase que, en base al AST, extraiga la información necesaria y aplique las validaciones pertinentes al  (2024-09-25, v0.2.0) Análisis Semántico , garantizando que el programa de entrada es correcto con respecto al dominio.

² Ver [Phoenix Server](#), de Martin Fowler (2021-07-10), e [Immutable Server](#), de Kief Morris (2013-06-13).

- (II). Una fase final de  (2024-05-16, v0.1.0) Generación de Código que produzca los artefactos finales de la solución teniendo en cuenta, según el dominio, la relevancia o no del  (2024-05-16, v0.1.0) Runtime .

Adicionalmente y por tratarse de la última etapa del proyecto, se deberá confeccionar un documento en formato PDF, [Notion](#) o [Confluence](#), que contenga como mínimo las siguientes secciones fundamentales:

Tabla de Contenidos

1. Introducción

2. Modelo Computacional

2.1. Dominio

2.2. Lenguaje

3. Implementación

3.1. Frontend

3.2. Backend

3.3. Adicionales (*opcional*)

3.4. Dificultades Encontradas

4. Futuras Extensiones

5. Conclusiones

6. Referencias

7. Bibliografía³

Se espera que cada sección describa lo referente al desarrollo y a la concepción de sus ideas, pero no que se explaye sobre conceptos teóricos o prácticos pertinentes a la Teoría de Lenguajes (*e.g.*, no es necesario explicar qué es un analizador sintáctico o una gramática).

5.2. FAQ

¿Puedo usar librerías externas?

Sí, en cuyo caso primero se deberá solicitar aprobación vía correo al QRF indicando la necesidad arquitectural y/o tecnológica de la misma. Luego de aprobada, se deberá referenciar en el README junto con su versión y en el informe final.

³ La bibliografía representa la lista de material consultado que no se referencia explícitamente dentro del documento. Por ejemplo, si en alguna oración se expresa “[...] véase Turing (1936)”, o también “[...] ver Aho et al. (2006)”, entonces dichas citas se deben incluir en la sección de **referencias**. Caso contrario, deberán ir en la sección bibliográfica.